



Extraer imágenes de un documento PDF

Para concluir, en este subtema veremos cómo podemos recuperar imágenes de un documento PDF, para esto, utilizaremos `pymupdf`, una herramienta que ya habíamos utilizado para extraer texto. Para este ejemplo, utilizaremos el documento denominado “campusPue”, mismo que podrás encontrar en los archivos adjuntos de la plataforma, este documento PDF se compone en un primer momento de texto, seguido de dos imágenes.

A continuación, tenemos un par de hermosas fotografías del Tecnológico de Monterrey campus Puebla, tomadas el 15 de abril de 2025



Para dar comienzo, importamos la biblioteca `pymupdf`.

```
!pip install pymupdf
```

A continuación, la importamos junto con otras dos bibliotecas que nos ayudarán a trabajar con las imágenes. La primera, **PIL.Image**, es una biblioteca para trabajar con imágenes que provee muchas funcionalidades. Nosotros solo la utilizaremos para guardar la imagen en nuestro directorio de trabajo, apoyándonos también en la biblioteca `io`, que forma parte de Python y nos permite trabajar con flujos de entrada y salida. En este caso, la utilizaremos para escribir nuestra imagen en un archivo.

```
import pymupdf as fitz
import PIL.Image
import io
```

Lo siguiente que haremos, es abrir nuestro archivo PDF, leer la única página disponible y asignarla a la variable `page`, luego le pedimos a `pymupdf` que extraiga las imágenes de esa página y las almacene en la variable `images`.

```
pdf = fitz.open("campusPue.pdf")
page = pdf[0]
images = page.get_images()
```

Ya que tenemos las imágenes, vamos a recorrerlas, para obtener la información de la imagen, y escribirlas en nuestro directorio de trabajo de la siguiente manera.

```
counter = 1

for image in images:

    base_img = pdf.extract_image(image[0])

    print (base_img)

    print("The image", counter, "has a resolution of ",

          base_img['width'], 'X', base_img['height'],

          "and is a", base_img['ext'], "image")

    image_data = base_img["image"]

    img = PIL.Image.open(io.BytesIO(image_data))

    ext = base_img["ext"]

    img.save(open(f"img{counter}.{ext}", "wb"))

    counter+=1;
```

Al ejecutar todas las celdas, observamos que se generan los archivos img1.jpeg e img2.jpeg.



```
[ ] counter = 1
for image in images:
    base_img = pdf.extract_image(image[0])
    print(base_img)
    print("The image", counter, "has a resolution of ",
          base_img['width'], 'X', base_img['height'],
          "and is a", base_img['ext'], "image")
    image_data = base_img["image"]
    img = PIL.Image.open(io.BytesIO(image_data))
    ext = base_img["ext"]
    img.save(open(f"img{counter}.{ext}", "wb"))
    counter+=1
```

```
{'width': 2077, 'height': 1169, 'ext': 'jpeg', 'colorspace': 3, 'xres': 96, 'yres': 96, 'bpc': 8, 'size': 813748, 'image': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00H\x00H\x00'}
The image 1 has a resolution of  2077 X 1169 and is a jpeg image
{'width': 1190, 'height': 670, 'ext': 'jpeg', 'colorspace': 3, 'xres': 96, 'yres': 96, 'bpc': 8, 'size': 240979, 'image': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00G\x00G\x00'}
The image 2 has a resolution of  1190 X 670 and is a jpeg image
```

Al analizar este fragmento de código, podemos ver que estamos recorriendo las imágenes con un ciclo `for`. Para cada una, extraemos un diccionario llamado `base_img`, que contiene información de la imagen como; la resolución, extensión o tipo de imagen, entre otros datos, incluida la información misma de la imagen. La variable `img` contiene la información de la imagen misma ya preparada para ser almacenada en forma de un archivo, la variable `ext` guarda la extensión del archivo que posteriormente se guarda con el nombre de `img{counter}.{ext}`.

Lo expuesto en este recurso es posible replicarlo para cada página de un documento que necesitemos trabajar, obteniendo así, todas las imágenes en su contenido.

| Glosario

PIL (Pillow - Python Imaging Library). Biblioteca para procesamiento de imágenes en Python. Permite abrir, manipular y guardar imágenes en diferentes formatos.

io (Input/Output). Módulo de Python que proporciona herramientas para manejar flujos de datos (bytes, archivos). En este caso, se usa para convertir datos binarios de imágenes en archivos guardables.

fitz.open(). Función de PyMuPDF que abre un archivo PDF para su procesamiento.

page.get_images(). Método que extrae una lista de imágenes presentes en una página específica del PDF.

Diccionario de imagen (base_img). Estructura que almacena metadatos de una imagen extraída, incluyendo:

- **width:** Ancho en píxeles.
- **height:** Alto en píxeles.
- **ext:** Extensión del formato de imagen (ej. "jpeg", "png").
- **image:** Datos binarios de la imagen.

PIL.Image.open(). Función de Pillow que crea un objeto de imagen a partir de datos binarios.

img.save(). Método de Pillow para guardar una imagen en un archivo.

Contador (counter). Variable para numerar imágenes guardadas.

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 30 de marzo del 2025 y su disponibilidad depende del autor.

| Material adicional

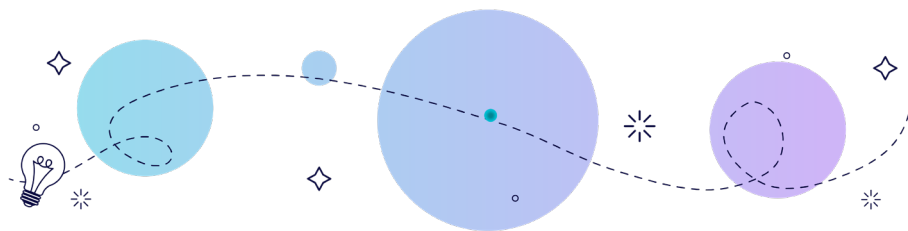
pdf2image. (s.f.). *pdf2image documentation* [Documentación de pdf2image]. Recuperado el 30 de marzo de 2024, de <https://pdf2image.readthedocs.io/en/latest/index.html>

Tabula-py. (s.f.). *Tabula-py documentation* [Documentación de Tabula-py]. Recuperado el 30 de marzo de 2025, de <https://tabula-py.readthedocs.io/en/latest/tabula.html>

Vine, J. (s.f.). *pdfplumber* [pdfplumber]. GitHub. Recuperado el 30 de marzo de 2024, de <https://github.com/jsvine/pdfplumber>

YouTube. (s.f.). [Video] *Working with PDFs in Python* [[Video] Trabajando con PDFs en Python]. Recuperado el 30 de marzo de 2025, de <https://www.youtube.com/watch?v=w2r2Bg42UPY>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 30 de marzo del 2025 y su disponibilidad depende del autor.



| Bibliografía

FreeCodeCamp. (s.f.). *Extract data from PDF files with Python* [Extraer datos de archivos PDF con Python]. Recuperado el 30 de marzo de 2025, de <https://www.freecodecamp.org/news/extract-data-from-pdf-files-with-python/>

PyMuPDF. (s.f.). *PyMuPDF documentation* [Documentación de PyMuPDF]. Recuperado el 30 de marzo de 2025, de <https://pymupdf.readthedocs.io/en/latest/>

PyMuPDF. (s.f.). *PyMuPDF tutorial* [Tutorial de PyMuPDF]. Recuperado el 30 de marzo de 2025, de <https://pymupdf.readthedocs.io/en/latest/tutorial.html>

PyMuPDF. (s.f.). *Recipes for working with images* [Recetas para trabajar con imágenes]. Recuperado el 30 de marzo de 2025, de <https://pymupdf.readthedocs.io/en/latest/recipes-images.html>

PyPDF. (s.f.). *PyPDF documentation* [Documentación de PyPDF]. Recuperado el 30 de marzo de 2025, de <https://pypdf.readthedocs.io/en/stable/>

PyPDF. (s.f.). *Extracting images from PDFs* [Extracción de imágenes de PDFs]. Recuperado el 30 de marzo de 2025, de <https://pypdf.readthedocs.io/en/stable/user/extract-images.html>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 30 de marzo del 2025 y su disponibilidad depende del autor.